# ฝึกเขียนโค้ด 8 ข้อ ก่อนสอบ

ตอนที่ 2 มี 2 แบบ: **2.1 เขียนคลาสจาก UML** (Inheritance · Polymorphism · Abstraction) และ **2.2 เติมโค้ดที่หายไป** (Exception · GUI) ทุกข้อในชุดนี้ดัดแปลงจากตัวอย่างและ workshop ในสไลด์ของอาจารย์ และเฉลยทุกข้อ compile + รันจริงแล้ว

### เขียนไม่ครบก็ได้คะแนน: ไล่เขียนตามลำดับนี้

1. **หัวคลาส** ให้ถูก: `public abstract class X` / `class Y extends X` / `class Z implements I`
2. **attribute** ตาม UML: `-` = `private`, `#` = `protected`, `+` = `public` · กลับด้านชนิด `name : String` → `String name;`
3. **constructor** ชื่อเดียวกับคลาส ไม่มี return type · คลาสลูกบรรทัดแรกต้องเป็น `super(...)` แล้วตามด้วย `this.x = x;`
4. **abstract method** ในแม่: ไม่มี body ลงท้าย `;` · ลูกเขียนให้ครบพร้อม `@Override`
5. **main**: `ArrayList<แม่>` + `add(new ลูก(...))` + for-each เรียกเมธอด
6. ข้อ GUI/Exception: นึกคู่คำให้ออก `try` ↔ `catch`, `parseInt/parseDouble` ↔ `NumberFormatException`, หาร ↔ `ArithmeticException`, `nextInt` ↔ `InputMismatchException`, ปุ่ม ↔ `addActionListener`

ถ้าลืม syntax บางตัว **อย่าเว้นว่าง** เขียนเท่าที่จำได้ แล้วใส่คอมเมนต์ `// คำนวณ เงินเดือน = base + bonus` ไว้ก็ยังแสดงว่าเข้าใจ

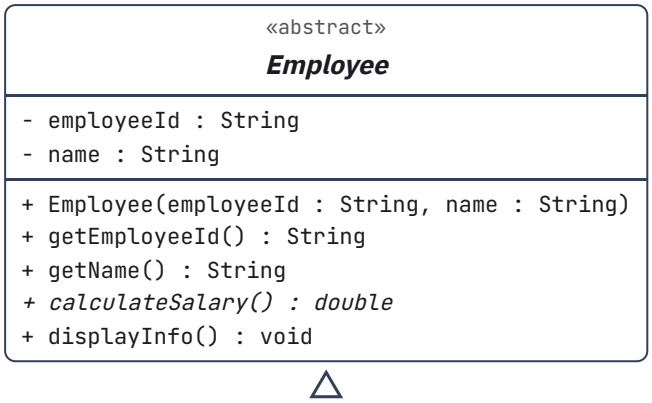
### เช็คลิสต์ก่อนส่ง

☐ ทุกบรรทัดมี `;` ☐ ปีกกาเปิด–ปิดครบ ☐ `super(...)` บรรทัดแรก ☐ เมธอดจาก interface มี `public` ☐ `@Override` ชื่อตรงกับแม่ ☐ `import` ครบ (ArrayList, Scanner, InputMismatchException, javax.swing, java.awt, java.awt.event) ☐ GUI จบด้วย `setVisible(true)`

วิธีใช้: ทำโจทย์ข้อ 1–8 ก่อนโดยไม่เปิดเฉลย จับเวลาข้อละ 10–15 นาที แล้วค่อยเปิดเฉลยท้ายเล่มตรวจทีละบรรทัด

# ข้อ 1 · ระบบเงินเดือนพนักงาน

Inheritance + Abstraction + Polymorphism




△

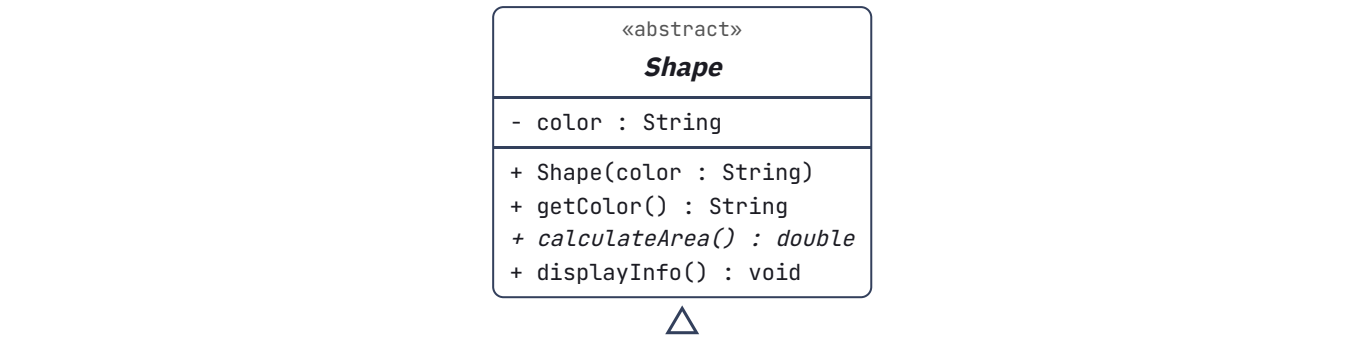
| FullTimeEmployee | PartTimeEmployee |
|---|---|
| - baseSalary : double<br>- bonus : double | - hoursWorked : int<br>- hourlyRate : double |
| + FullTimeEmployee(employeeId, name, baseSalary, bonus)<br>+ calculateSalary() : double | + PartTimeEmployee(employeeId, name, hoursWorked, hourlyRate)<br>+ calculateSalary() : double |

- `calculateSalary()` ของ FullTime = baseSalary + bonus · ของ PartTime = hoursWorked × hourlyRate
- `displayInfo()` พิมพ์ `ID: … Name: … Salary: …` (เรียก `calculateSalary()`)
- เขียน `main` : สร้าง `ArrayList<Employee>` ใส่ FullTime("E001","Anan",30000,5000) และ PartTime("E002","Suda",80,200) วนพิมพ์ข้อมูล และรวมเงินเดือนทั้งหมด

> **ผลลัพธ์ที่ต้องได้**
>
> ```
> ID: E001 Name: Anan Salary: 35000.0
> ID: E002 Name: Suda Salary: 16000.0
> Total: 51000.0
> ```

# ข้อ 2 · รูปทรงเรขาคณิต

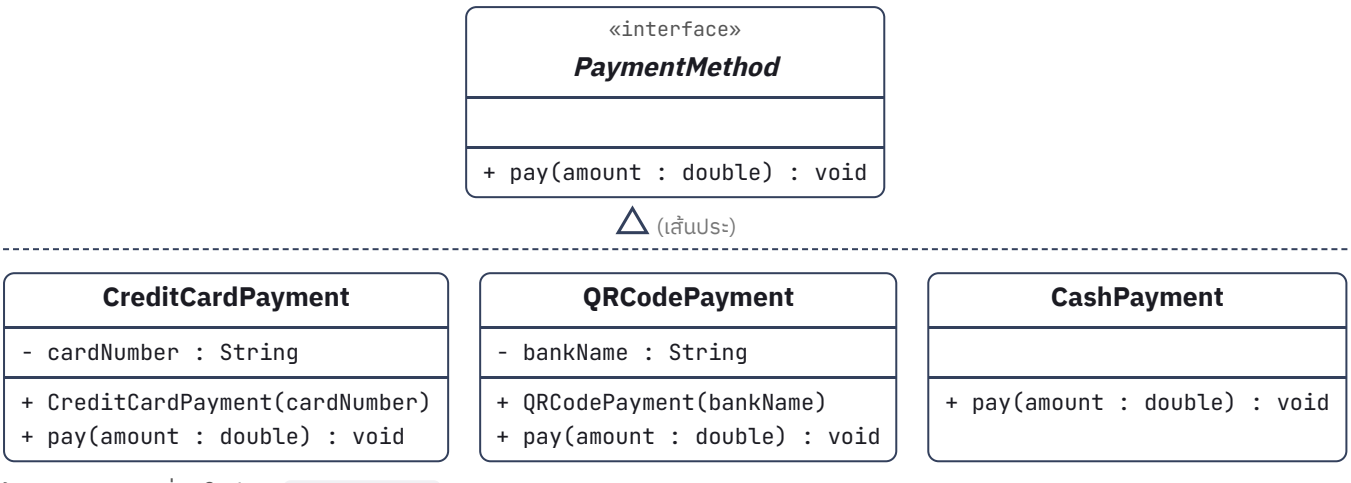
Abstract class + Polymorphism ผ่าน array




- พื้นที่: วงกลม = π·r² ( `Math.PI` ) · สี่เหลี่ยม = กว้าง × สูง · สามเหลี่ยม = ½ × ฐาน × สูง
- `displayInfo()` พิมพ์ `Color: …` และ `Area: …` ทศนิยม 2 ตำแหน่ง ( `printf("%.2f%n", …)` )
- `main` : `Shape[]` 3 ตัว = Circle("Red",5), Rectangle("Blue",4,6), Triangle("Green",8,3) วนแสดงผลและรวมพื้นที่

> **ผลลัพธ์บรรทัดสุดท้าย**
>
> `Total area: 114.54`

## ข้อ 3 · ระบบชำระเงิน Interface + implements + Polymorphism

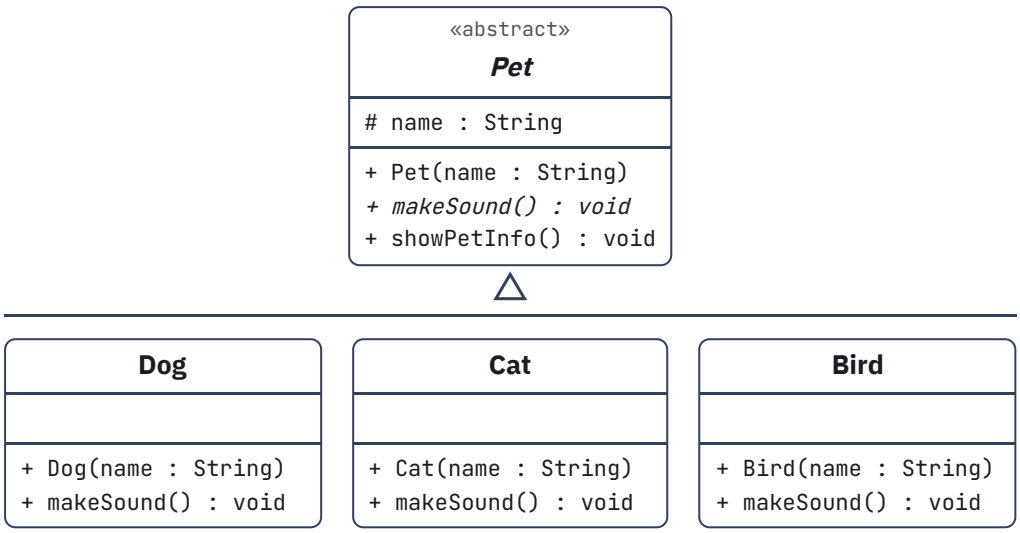



เส้น**ประ**หัวสามเหลี่ยมโปร่ง = `implements`

- เขียน interface และ 3 คลาส · เมธอด `pay` พิมพ์ตามตัวอย่าง
- เขียนเมธอด `static void processPayment(PaymentMethod p, double amount)` ที่เรียก `p.pay(amount)` แล้วใช้ใน main

> **ผลลัพธ์ที่ต้องได้**
>
> ```
> Paid 1500.00 baht by credit card 1234
> Paid 750.00 baht by QR Code via ABC Bank
> Paid 300.00 baht by cash
> ```

## ข้อ 4 · ระบบสัตว์เลี้ยง protected (#) + abstract + ArrayList




- `showPetInfo()` พิมพ์ `ชื่อ + " Making Sound : "` (ไม่ขึ้นบรรทัด) แล้วเรียก `makeSound()`
- Dog → `Bok Bok...` · Cat → `Meaow Meaow...` · Bird → `Jib Jib...`
- `main` : `ArrayList<Pet>` ใส่ Dog("Buddy"), Cat("Mimi"), Bird("Tweety") แล้ววนเรียก showPetInfo()

> **ผลลัพธ์ที่ต้องได้**
>
> ```
> Buddy Making Sound : Bok Bok...
> Mimi Making Sound : Meaow Meaow...
> Tweety Making Sound : Jib Jib...
> ```

## ข้อ 5 · เครื่องคิดเลขหาร

try · catch หลายตัว · finally

เติมช่อง (1)–(8) ให้โปรแกรมทำงานตามตาราง

```
import java.util.Scanner;
import java.util.(1)          ;
public class Q5 {
    public static void main(String[] args) {
        (2)          sc = new Scanner(System.in);
        (3)          {
            System.out.print("Enter number 1: ");
            int a = sc.(4)          ();
            System.out.print("Enter number 2: ");
            int b = sc.nextInt();
            int result = a / b;
            System.out.println("Result = " + result);
        } catch ((5)          e) {
            System.out.println("Error: Cannot divide by zero");
        } (6)          ((7)          e) {
            System.out.println("Error: Please enter a number");
        } (8)          {
            sc.close();
        }
        System.out.println("End Program");
    }
}
```

| ป้อน | ผลลัพธ์ |
|---|---|
| 10, 2 | Result = 5 → End Program |
| 10, 0 | Error: Cannot divide by zero → End Program |
| ABC | Error: Please enter a number → End Program (ไม่ถามเลขที่ 2) |

ถาม-ต่อ: ทำไมป้อน ABC แล้วไม่ถามเลขที่ 2? (ตอบ: เกิด exception แล้วข้ามบรรทัดที่เหลือใน try ทันที)

## ข้อ 6 · บัญชีธนาคาร

throw · throws · Custom Exception

เติมช่อง (1)–(8)

```
class InsufficientBalanceException (1)          {
    public InsufficientBalanceException(String message) {
        (2)          (message);
    }
}
class BankAccount {
    private double balance;
    public BankAccount(double balance) {
        if (balance < 0) {
            (3)           IllegalArgumentException("Balance cannot be negative");
        }
        this.balance = balance;
    }
    public void withdraw(double amount) (4)          InsufficientBalanceException {
        if (amount > balance) {
            (5)          ("Insufficient balance");
        }
        balance -= amount;
    }
    public double getBalance() { return balance; }
}
public class Q6 {
    public static void main(String[] args) {
        BankAccount acc = new BankAccount(1000);
        try {
            acc.withdraw(300);
            System.out.println("Balance: " + acc.getBalance());
            acc.withdraw(5000);
            System.out.println("This line is skipped");
        } catch ((6)           e) {
            System.out.println("Error: " + e.(7)          ());
        } (8)          {
            System.out.println("Thank you");
        }
    }
}
```

**ผลลัพธ์ที่ต้องได้**

```
Balance: 700.0
Error: Insufficient balance
Thank you
```

จำ: **throw** = โยนในเนื้อเมธอด (ตามด้วย `new`) · **throws** = ประกาศที่หัวเมธอด · custom exception ที่ `extends Exception` เป็น checked จึงต้อง throws + try-catch

## ข้อ 7 · โปรแกรมบวกเลข GUI

Layout · Event · try-catch ใน GUI

เติมช่อง (1)–(16) · ฟอร์ม 2 แถว 2 คอลัมน์อยู่ด้านบน ผลลัพธ์อยู่ตรงกลาง ปุ่มอยู่ด้านล่าง · ถ้ากรอกตัวอักษรต้องเด้งกล่องข้อความแจ้ง error

```
import javax.swing.*;
import java.awt.(1)          ;
import java.awt.GridLayout;
public class Q7 (2)          {
    private JTextField firstField, secondField;
    private JLabel resultLabel;
    public Q7() {
        firstField = new JTextField(10);
        secondField = new JTextField(10);
        JButton calculateButton = new JButton("คำนวณ");
        resultLabel = new JLabel("ผลลัพธ์จะแสดงที่นี่", SwingConstants.CENTER);
        JPanel formPanel = new JPanel(new (3)          (2, 2, 5, 5));
        formPanel.add(new JLabel("จำนวนที่ 1:"));
        formPanel.(4)          (firstField);
        formPanel.add(new JLabel("จำนวนที่ 2:"));
        formPanel.add(secondField);
        setLayout(new BorderLayout(10, 10));
        add(formPanel, BorderLayout.(5)          );
        add(resultLabel, BorderLayout.CENTER);
        add(calculateButton, BorderLayout.(6)          );
        calculateButton.(7)          (e -> calculate());
        setTitle("โปรแกรมบวกเลข");
        setSize(400, 220);
        setDefaultCloseOperation(JFrame.(8)          );
        setLocationRelativeTo(null);
    }
    private void calculate() {
        (9)          {
            double a = Double.(10)          (firstField.(11)          ().trim());
            double b = Double.parseDouble(secondField.getText().trim());
            resultLabel.(12)          ("ผลลัพธ์ = " + (a + b));
        } catch ((13)          e) {
            JOptionPane.(14)          (this, "กรุณากรอกข้อมูลเป็นตัวเลข",
                    "ข้อมูลไม่ถูกต้อง", JOptionPane.(15)          );
        }
    }
    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> new Q7().(16)          (true));
    }
}
```

## ข้อ 8 · BMI Calculator

แยก Model/UI · anonymous class · 2 catch

เติมช่อง (1)–(11) · BMI = น้ำหนัก(kg) / ส่วนสูง(m)² · <18.5 Underweight · 18.5–24.9 Normal · 25–29.9 Overweight · ≥30 Obese · ค่า ≤ 0 ให้ throw · 65 kg / 170 cm → **22.49 Normal**

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.ActionEvent;
import java.awt.event.(1)           ;
class BMI {
    private double weight, heightCm;
    public BMI(double weight, double heightCm) {
        if (weight <= 0 || heightCm <= 0) (2)           new IllegalArgumentException("ค่าต้องมากกว่า 0");
        this.weight = weight; this.heightCm = heightCm;
    }
    public double calculate() { double m = heightCm / 100; return (3)           ; }
    public String getResult() {
        double b = calculate();
        if (b < (4)          ) return "Underweight";
        else if (b < 25) return "Normal";
        else if (b < 30) return "Overweight";
        else return "Obese";
    }
}
public class Q8 extends JFrame {
    private JTextField weightField = new JTextField(10), heightField = new JTextField(10);
    private JLabel resultLabel = new JLabel("BMI: -", SwingConstants.CENTER);
    public Q8() {
        JButton calcButton = new JButton("Calculate");
        JPanel form = new JPanel(new GridLayout(2, 2, 5, 5));
        form.add(new JLabel("Weight (kg):")); form.add(weightField);
        form.add(new JLabel("Height (cm):")); form.add(heightField);
        setLayout(new BorderLayout(10, 10));
        add(form, BorderLayout.NORTH);
        add(resultLabel, BorderLayout.CENTER);
        add(calcButton, BorderLayout.SOUTH);
        calcButton.addActionListener((5)           {
            @Override
            public void (6)          ((7)           e) {
                try {
                    double w = Double.parseDouble(weightField.getText().trim());
                    double h = Double.parseDouble(heightField.getText().trim());
                    BMI bmi = (8)          ;
                    resultLabel.setText(String.format("BMI: %.2f  Result: %s", bmi.calculate(),
bmi.getResult()));
                } catch ((9)           ex) {
                    JOptionPane.showMessageDialog(Q8.this, "กรุณากรอกตัวเลข", "Error",
JOptionPane.ERROR_MESSAGE);
                } catch ((10)          ex) {
                    JOptionPane.showMessageDialog(Q8.this, ex.(11)          (), "Error",
JOptionPane.WARNING_MESSAGE);
                }
            }
        });
        setTitle("BMI Calculator"); setSize(350, 200);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE); setLocationRelativeTo(null);
    }
    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> new Q8().setVisible(true));
    }
}
```

# เฉลย

## ข้อ 1 · Employee

จุดให้คะแนน: `abstract class` + abstract method ไม่มี body · ลูก `extends` · `super(employeeId, name)` บรรทัดแรก · `@Override calculateSalary` ครบ 2 คลาส · main ใช้ตัวแปรชนิดแม่ (polymorphism)

```
import java.util.ArrayList;
abstract class Employee {
    private String employeeId;
    private String name;
    public Employee(String employeeId, String name) {
        this.employeeId = employeeId;
        this.name = name;
    }
    public String getEmployeeId() { return employeeId; }
    public String getName() { return name; }
    public abstract double calculateSalary();
    public void displayInfo() {
        System.out.println("ID: " + employeeId + " Name: " + name + " Salary: " +
calculateSalary());
    }
}
class FullTimeEmployee extends Employee {
    private double baseSalary;
    private double bonus;
    public FullTimeEmployee(String employeeId, String name, double baseSalary, double bonus) {
        super(employeeId, name);
        this.baseSalary = baseSalary;
        this.bonus = bonus;
    }
    @Override
    public double calculateSalary() { return baseSalary + bonus; }
}
class PartTimeEmployee extends Employee {
    private int hoursWorked;
    private double hourlyRate;
    public PartTimeEmployee(String employeeId, String name, int hoursWorked, double hourlyRate) {
        super(employeeId, name);
        this.hoursWorked = hoursWorked;
        this.hourlyRate = hourlyRate;
    }
    @Override
    public double calculateSalary() { return hoursWorked * hourlyRate; }
}
public class Q1 {
    public static void main(String[] args) {
        ArrayList<Employee> list = new ArrayList<>();
        list.add(new FullTimeEmployee("E001", "Anan", 30000, 5000));
        list.add(new PartTimeEmployee("E002", "Suda", 80, 200));
        double total = 0;
        for (Employee e : list) {
            e.displayInfo();
            total += e.calculateSalary();
        }
        System.out.println("Total: " + total);
    }
}
```

## ข้อ 2 · Shape

จุดให้คะแนน: `Shape s = new Circle(...)` ได้แต่ `new Shape(...)` ไม่ได้ · ลูกทุกตัวเรียก `super(color)` · สูตรพื้นที่ถูก · ลูป `for (Shape s : shapes)`

```
abstract class Shape {
    private String color;
    public Shape(String color) { this.color = color; }
    public String getColor() { return color; }
    public abstract double calculateArea();
    public void displayInfo() {
        System.out.println("Color: " + color);
        System.out.printf("Area: %.2f%n", calculateArea());
    }
}
class Circle extends Shape {
    private double radius;
    public Circle(String color, double radius) { super(color); this.radius = radius; }
    @Override
    public double calculateArea() { return Math.PI * radius * radius; }
}
class Rectangle extends Shape {
    private double width, height;
    public Rectangle(String color, double width, double height) {
        super(color); this.width = width; this.height = height;
    }
    @Override
    public double calculateArea() { return width * height; }
}
class Triangle extends Shape {
    private double base, height;
    public Triangle(String color, double base, double height) {
        super(color); this.base = base; this.height = height;
    }
    @Override
    public double calculateArea() { return 0.5 * base * height; }
}
public class Q2 {
    public static void main(String[] args) {
        Shape[] shapes = { new Circle("Red", 5.0), new Rectangle("Blue", 4.0, 6.0), new
Triangle("Green", 8.0, 3.0) };
        double total = 0;
        for (Shape s : shapes) { s.displayInfo(); total += s.calculateArea(); }
        System.out.printf("Total area: %.2f%n", total);
    }
}
```

### ข้อ 3 · PaymentMethod

จุดให้คะแนน: `interface` เมธอดไม่มี body · คลาสใช้ `implements` · เมธอด `pay` ต้องมี `public` (ไม่ใส่ = compile error) · พารามิเตอร์ชนิด interface (program to an interface)

```
interface PaymentMethod {
    void pay(double amount);
}
class CreditCardPayment implements PaymentMethod {
    private String cardNumber;
    public CreditCardPayment(String cardNumber) { this.cardNumber = cardNumber; }
    @Override
    public void pay(double amount) {
        System.out.printf("Paid %.2f baht by credit card %s%n", amount, cardNumber);
    }
}
class QRCodePayment implements PaymentMethod {
    private String bankName;
```

```
    public QRCodePayment(String bankName) { this.bankName = bankName; }
    @Override
    public void pay(double amount) {
        System.out.printf("Paid %.2f baht by QR Code via %s%n", amount, bankName);
    }
}
class CashPayment implements PaymentMethod {
    @Override
    public void pay(double amount) {
        System.out.printf("Paid %.2f baht by cash%n", amount);
    }
}
public class Q3 {
    public static void processPayment(PaymentMethod p, double amount) { p.pay(amount); }
    public static void main(String[] args) {
        processPayment(new CreditCardPayment("1234"), 1500);
        processPayment(new QRCodePayment("ABC Bank"), 750);
        processPayment(new CashPayment(), 300);
    }
}
```

## ข้อ 4 · Pet

จุดให้คะแนน: `#` = `protected` · `import java.util.ArrayList;` · ลูกเรียก `super(name)` · เมธอดแม่เรียก `makeSound()` ได้แม้เป็น abstract (เพราะ polymorphism)

```
import java.util.ArrayList;
abstract class Pet {
    protected String name;
    public Pet(String name) { this.name = name; }
    public abstract void makeSound();
    public void showPetInfo() {
        System.out.print(name + " Making Sound : ");
        makeSound();
    }
}
class Dog extends Pet {
    public Dog(String name) { super(name); }
    @Override public void makeSound() { System.out.println("Bok Bok..."); }
}
class Cat extends Pet {
    public Cat(String name) { super(name); }
    @Override public void makeSound() { System.out.println("Meaow Meaow..."); }
}
class Bird extends Pet {
    public Bird(String name) { super(name); }
    @Override public void makeSound() { System.out.println("Jib Jib..."); }
}
public class Q4 {
    public static void main(String[] args) {
        ArrayList<Pet> pets = new ArrayList<>();
        pets.add(new Dog("Buddy"));
        pets.add(new Cat("Mimi"));
        pets.add(new Bird("Tweety"));
        for (Pet p : pets) p.showPetInfo();
    }
}
```

## ข้อ 5 · เครื่องคิดเลข (เฉลยช่อง)

ช่อง: (1) `InputMismatchException` · (2) `Scanner` · (3) `try` · (4) `nextInt` · (5) `ArithmeticException` · (6) `catch` · (7) `InputMismatchException` · (8) `finally`

```java
import java.util.Scanner;
import java.util.InputMismatchException;
public class Q5 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        try {
            System.out.print("Enter number 1: ");
            int a = sc.nextInt();
            System.out.print("Enter number 2: ");
            int b = sc.nextInt();
            int result = a / b;
            System.out.println("Result = " + result);
        } catch (ArithmeticException e) {
            System.out.println("Error: Cannot divide by zero");
        } catch (InputMismatchException e) {
            System.out.println("Error: Please enter a number");
        } finally {
            sc.close();
        }
        System.out.println("End Program");
    }
}
```

## ข้อ 6 · BankAccount (เฉลยช่อง)

ช่อง: (1) `extends Exception` · (2) `super` · (3) `throw new` · (4) `throws` · (5) `throw new InsufficientBalanceException` · (6) `InsufficientBalanceException` · (7) `getMessage` · (8) `finally`

```java
class InsufficientBalanceException extends Exception {
    public InsufficientBalanceException(String message) {
        super(message);
    }
}
class BankAccount {
    private double balance;
    public BankAccount(double balance) {
        if (balance < 0) {
            throw new IllegalArgumentException("Balance cannot be negative");
        }
        this.balance = balance;
    }
    public void withdraw(double amount) throws InsufficientBalanceException {
        if (amount > balance) {
            throw new InsufficientBalanceException("Insufficient balance");
        }
        balance -= amount;
    }
    public double getBalance() { return balance; }
}
public class Q6 {
    public static void main(String[] args) {
        BankAccount acc = new BankAccount(1000);
        try {
            acc.withdraw(300);
            System.out.println("Balance: " + acc.getBalance());
            acc.withdraw(5000);
            System.out.println("This line is skipped");
        } catch (InsufficientBalanceException e) {
            System.out.println("Error: " + e.getMessage());
        } finally {
            System.out.println("Thank you");
        }
```

```
    }
}
```

### ข้อ 7 · AdditionGUI (เฉลยช่อง)

ช่อง: (1) `BorderLayout` · (2) `extends JFrame` · (3) `GridLayout` · (4) `add` · (5) `NORTH` · (6) `SOUTH` · (7) `addActionListener` · (8) `EXIT_ON_CLOSE` · (9) `try` · (10) `parseDouble` · (11) `getText` · (12) `setText` · (13) `NumberFormatException` · (14) `showMessageDialog` · (15) `ERROR_MESSAGE` · (16) `setVisible`

```
import javax.swing.*;
import java.awt.BorderLayout;
import java.awt.GridLayout;
public class Q7 extends JFrame {
    private JTextField firstField, secondField;
    private JLabel resultLabel;
    public Q7() {
        firstField = new JTextField(10);
        secondField = new JTextField(10);
        JButton calculateButton = new JButton("คำนวณ");
        resultLabel = new JLabel("ผลลัพธ์จะแสดงที่นี่", SwingConstants.CENTER);
        JPanel formPanel = new JPanel(new GridLayout(2, 2, 5, 5));
        formPanel.add(new JLabel("จำนวนที่ 1:"));
        formPanel.add(firstField);
        formPanel.add(new JLabel("จำนวนที่ 2:"));
        formPanel.add(secondField);
        setLayout(new BorderLayout(10, 10));
        add(formPanel, BorderLayout.NORTH);
        add(resultLabel, BorderLayout.CENTER);
        add(calculateButton, BorderLayout.SOUTH);
        calculateButton.addActionListener(e -> calculate());
        setTitle("โปรแกรมบวกเลข");
        setSize(400, 220);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLocationRelativeTo(null);
    }
    private void calculate() {
        try {
            double a = Double.parseDouble(firstField.getText().trim());
            double b = Double.parseDouble(secondField.getText().trim());
            resultLabel.setText("ผลลัพธ์ = " + (a + b));
        } catch (NumberFormatException e) {
            JOptionPane.showMessageDialog(this, "กรุณากรอกข้อมูลเป็นตัวเลข",
                    "ข้อมูลไม่ถูกต้อง", JOptionPane.ERROR_MESSAGE);
        }
    }
    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> new Q7().setVisible(true));
    }
}
```

### ข้อ 8 · BMI (เฉลยช่อง)

ช่อง: (1) `ActionListener` · (2) `throw` · (3) `weight / (m * m)` · (4) `18.5` · (5) `new ActionListener()` · (6) `actionPerformed` · (7) `ActionEvent` · (8) `new BMI(w, h)` · (9) `NumberFormatException` · (10) `IllegalArgumentException` · (11) `getMessage`
สังเกต: catch `NumberFormatException` ต้องอยู่**ก่อน** `IllegalArgumentException` เพราะเป็นคลาสลูกของมัน (สลับ = compile error)

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.ActionEvent;
```

```
import java.awt.event.ActionListener;
class BMI {
    private double weight, heightCm;
    public BMI(double weight, double heightCm) {
        if (weight <= 0 || heightCm <= 0) throw new IllegalArgumentException("ค่าต้องมากกว่า 0");
        this.weight = weight; this.heightCm = heightCm;
    }
    public double calculate() { double m = heightCm / 100; return weight / (m * m); }
    public String getResult() {
        double b = calculate();
        if (b < 18.5) return "Underweight";
        else if (b < 25) return "Normal";
        else if (b < 30) return "Overweight";
        else return "Obese";
    }
}
public class Q8 extends JFrame {
    private JTextField weightField = new JTextField(10), heightField = new JTextField(10);
    private JLabel resultLabel = new JLabel("BMI: -", SwingConstants.CENTER);
    public Q8() {
        JButton calcButton = new JButton("Calculate");
        JPanel form = new JPanel(new GridLayout(2, 2, 5, 5));
        form.add(new JLabel("Weight (kg):")); form.add(weightField);
        form.add(new JLabel("Height (cm):")); form.add(heightField);
        setLayout(new BorderLayout(10, 10));
        add(form, BorderLayout.NORTH);
        add(resultLabel, BorderLayout.CENTER);
        add(calcButton, BorderLayout.SOUTH);
        calcButton.addActionListener(new ActionListener() {
            @Override
            public void actionPerformed(ActionEvent e) {
                try {
                    double w = Double.parseDouble(weightField.getText().trim());
                    double h = Double.parseDouble(heightField.getText().trim());
                    BMI bmi = new BMI(w, h);
                    resultLabel.setText(String.format("BMI: %.2f  Result: %s", bmi.calculate(),
bmi.getResult()));
                } catch (NumberFormatException ex) {
                    JOptionPane.showMessageDialog(Q8.this, "กรุณากรอกตัวเลข", "Error",
JOptionPane.ERROR_MESSAGE);
                } catch (IllegalArgumentException ex) {
                    JOptionPane.showMessageDialog(Q8.this, ex.getMessage(), "Error",
JOptionPane.WARNING_MESSAGE);
                }
            }
        });
        setTitle("BMI Calculator"); setSize(350, 200);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE); setLocationRelativeTo(null);
    }
    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> new Q8().setVisible(true));
    }
}
```